

Onramp test instructions

Drafted with Claude (Anthropic) for Dr. Soule's lab – reviewed by Dax Soule.

Onramp test instructions – second run

Hi Neftali! You're the second human tester of the lab's new onboarding course (Rose ran the "beginner" profile; you're running the "mixed-skills" one). The course runs as a guided conversation with Claude inside a copy of the course repository, on your JupyterHub seat.

The big idea: you'll play a student with solid shell/Python/geology skills but **no pandas experience**. We're testing whether the course is smart enough to SKIP what you already know and properly teach the one thing you don't – not testing you. Your scripted answers below define that skill profile exactly; stick to them.

Time: plan for 2–3 hours total. **You do not need to do it in one sitting** – stopping and resuming is itself something we're testing (see section 6).

1. Setup (about 15 min, JupyterLab terminal)

```
git clone -b 001-coding-onramp-v1 https://github.com/qc-soule-lab/student-onramp.git
cd student-onramp
uv sync
claude
```

Inside Claude, type `/model claude-sonnet-4-6` (important – this exact model is what we’re validating; if your seat doesn’t offer it, stop and tell Dax). Then type:

```
assess me
```

When it asks your name: give your first name or say you’d rather stay anonymous – either is fine, and everything personal stays in your local copy (never uploaded, never committed).

2. The assessment – answer from this script, verbatim

Claude will ask about nine questions. **Answer each with the scripted line below.** Most of your answers are deliberately strong; the last three have deliberate gaps – keep them gappy even if you actually know the material.

1. **Terminal commands to find a file** – say: *“cd data, then ls -lh to see it and its size.”*
2. **Peeking at the top of a CSV** – say: *“head plate_boundary.csv – or head -3 if I just want the header and a couple of rows.”*
3. **A “No such file or directory” error** – say: *“I’m in the wrong directory – pwd to check where I am, then cd over or just use the full path.”*
4. **The vi editor** – say: *“vi file, it opens in normal mode; :q to quit, :q! to discard, :wq to save.”*
5. **Kilometers per degree** – say: *“1° lat ≈ 111 km; 1° lon is shorter, about $111 \cdot \cos(46^\circ) \approx 77$ km.”*
6. **Matplotlib / plotting code** – say: *“plt.imshow(depth) then plt.colorbar(label='depth (m)').”*
7. **Why there’s a volcano at Axial** – say: *“It’s a spreading ridge so there’s volcanism... not sure why OOI specifically.”* If it nudges you: *“ah – because it erupts so often, and a cable gives you real-time data instead of a yearly ship visit?”*

8. **xarray / opening a data file** – say: “I’d use xarray... *open_dataset? not sure how to see the variables.*” After it gives a concrete hint: “ah, `.data_vars`.”
9. **pandas / resample** – say: “I think `.resample` but I don’t really know pandas or what the 12-hour thing is.” Then let it walk you through its hints – stay a pandas beginner.

3. What SHOULD happen next – and what to watch for

After the assessment, Claude builds your plan. Given your script, the right plan is:

- **Steps 0 and 1 (shell/vi, mapping): capstone-only** – it should pose one confirmation task per step and move on. **If it makes you read chapters or do warm-up exercises for material you aced, that’s a finding – note it (but play along).**
- **Steps 2 and 3 (geology/00I, xarray): compressed** – a short exercise + capstone, no chapter reading.
- **Step 4 (pandas, the tide): full scaffold** – it should give you a book chapter to read, a guided exercise with hints, then the capstone. **If it skimps here, that’s also a finding.**

During the course: work normally, but **stay in character** – fluent on steps 0–3, genuine pandas beginner on step 4. Don’t volunteer pandas knowledge even if you have it. The vi capstone you can just do cleanly (open, read the header, :q) – no scripted failure for you.

4. The feedback questions – answer as YOURSELF

After every step, Claude asks three quick questions (difficulty / friction / keep-or-cut). **These are NOT scripted – give your genuine reactions.** This is the most valuable thing you produce: real human friction notes, especially “it went too slow / too fast for what I knew.”

5. Take notes outside the conversation too

Keep a scratch note of: anything confusing or broken, anywhere it taught you something you already knew (over-teaching is the failure mode YOUR profile exists to catch), how long the step-4 chapter actually took you to read, and anywhere it felt condescending or rushed.

6. Stopping and resuming

Stop whenever you like – just close the terminal. To resume (even days later): open a terminal, `cd student-onramp`, run `claude`, and type `continue`. It should greet you, tell you exactly where you left off, and **never re-ask an assessment question it already asked**. If it does re-ask one, note it – that’s a bug we want to know about.

7. When you finish

At the end, Claude will ask whether you’re willing to share your feedback file with Dax – please say yes and follow its instructions. Then also send Dax:

1. The conversation transcript: type `/export` in Claude and send the file it produces.
2. From your student-onramp folder: `feedback.md`, `journey_plan.md`, `progress.md` (these exist only on your seat – nothing is shared unless you send it).
3. Your scratch notes from section 5, photos fine.

8. If something breaks

Note what you typed and what happened (screenshot if easy), then either ask Claude to continue or stop and resume later. Breakage is a finding, not a failure – don’t troubleshoot for more than a couple of minutes. Questions any time: dax.soule@qc.cuny.edu.

Thank you, Neftali – your run is the one that proves the course respects what a student already knows.